



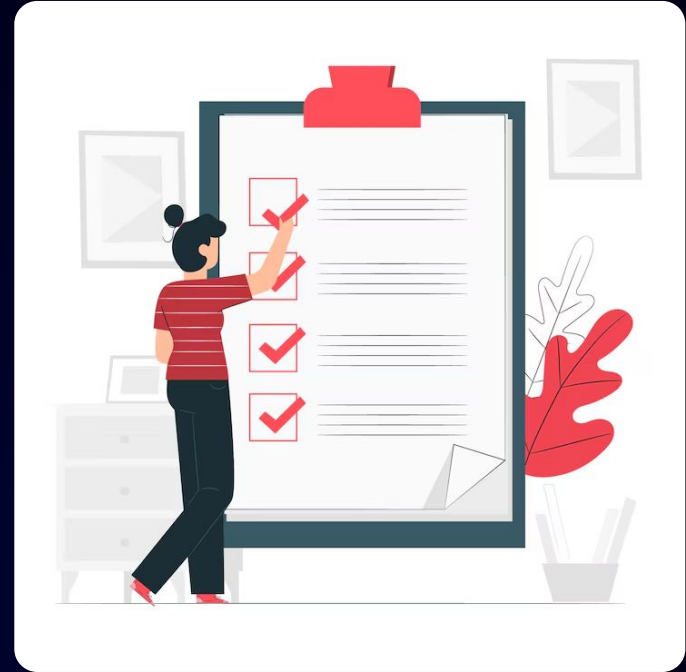
Introduction to Version Control & Git Basics

Learning Objectives

- Understand the importance of version control in software development
- Learn why version control is essential for tracking and managing changes
- Get introduced to Git fundamentals
- Create and initialize a Git repository
- Track changes in files using Git
- Make commits to save project progress

Prerequisites

- Basic understanding of programming concepts
- Familiarity with file and folder structure (directories)
- Basic knowledge of using a computer terminal/command line
- Understanding of how files are created, edited, and saved
- Willingness to learn and experiment with new tools

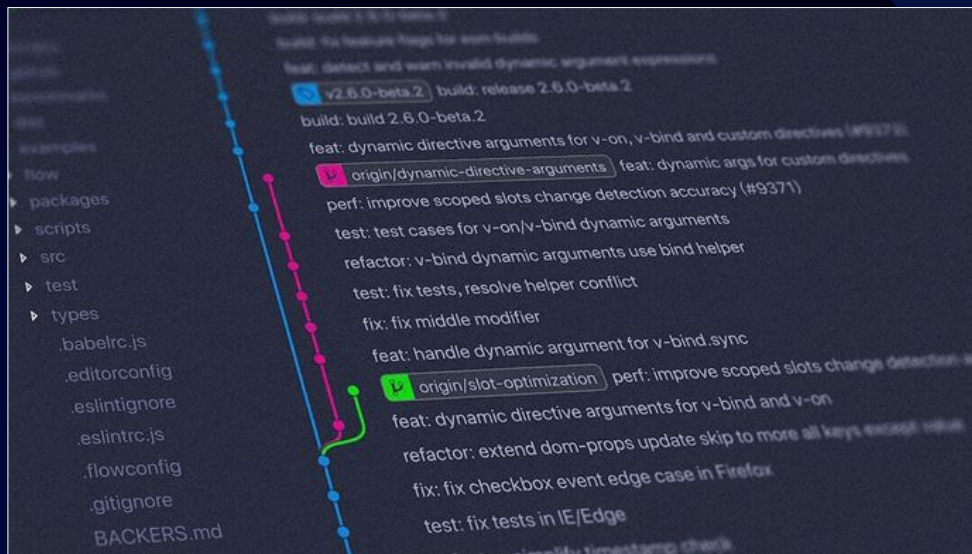




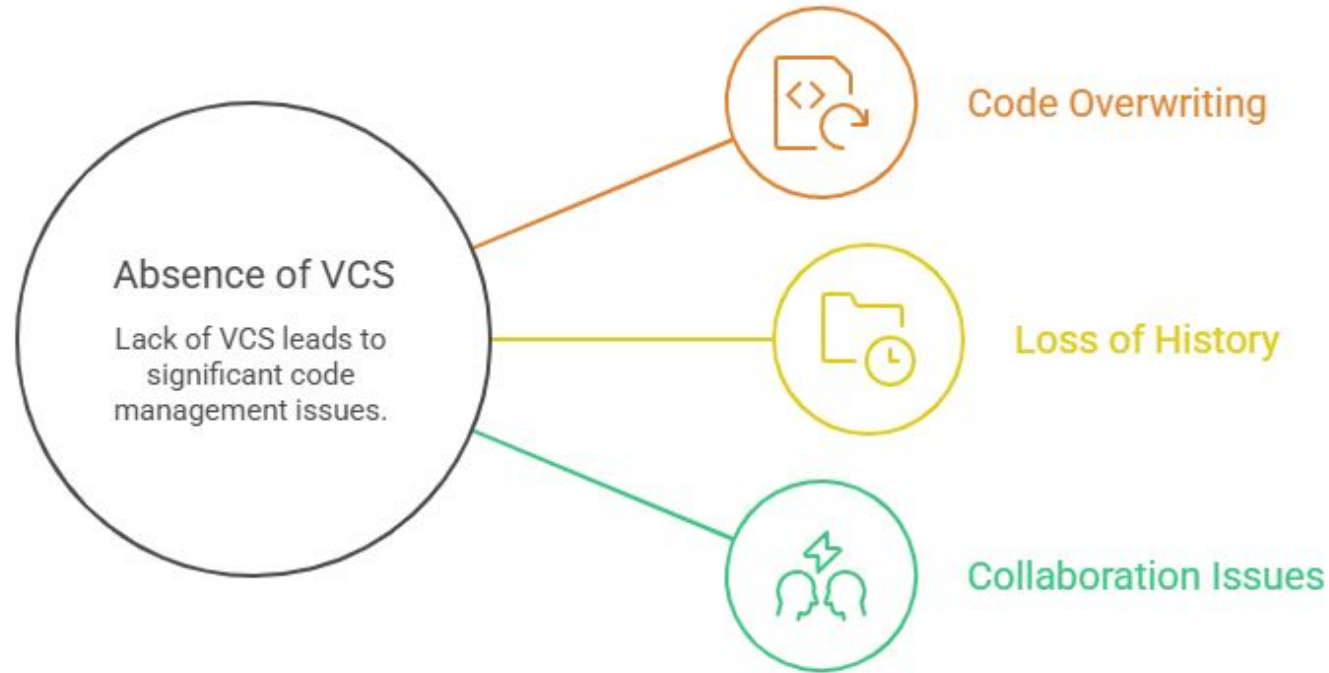
What is Version Control & Why It Matters

Introduction to Version Control Systems

- **Definition:** Version control systems (VCS) are tools that help manage changes to source code over time.
- **Importance:** VCS are crucial for maintaining code history, enabling collaborative development, and ensuring project integrity.



Unveiling the Challenges of VCS Absence



Benefits of Version Control Systems

- **Tracking Changes:** VCS tracks every modification made to the code, providing a detailed history of changes.
- **Reverting to Previous Versions:** Allows developers to revert to earlier versions of the codebase, helping to recover from errors.
- **Managing Multiple Versions:** Facilitates handling different versions of the code, such as release versions and experimental branches.



Pop Quiz

Q. What happens if Version Control is NOT used?

A

**Changes can be lost or
overwritten**

B

It is easier to manage changes

Pop Quiz

Q. What happens if Version Control is NOT used?

A

**Changes can be lost or
overwritten**

B

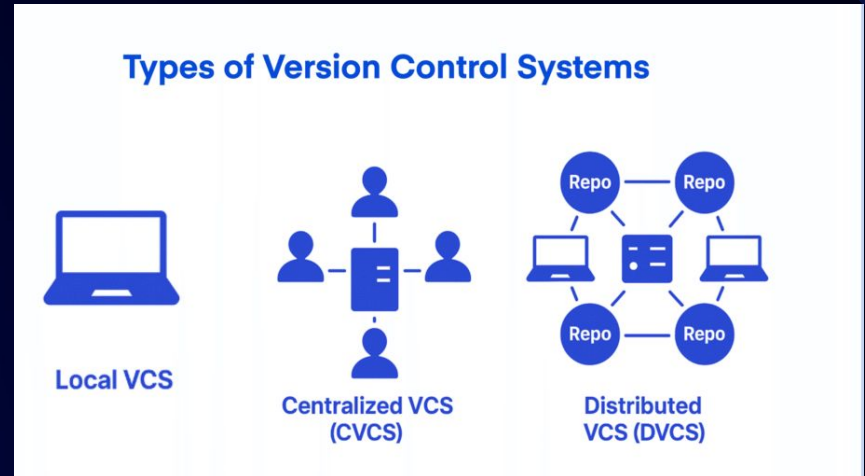
It is easier to manage changes



Types of Version Control Systems

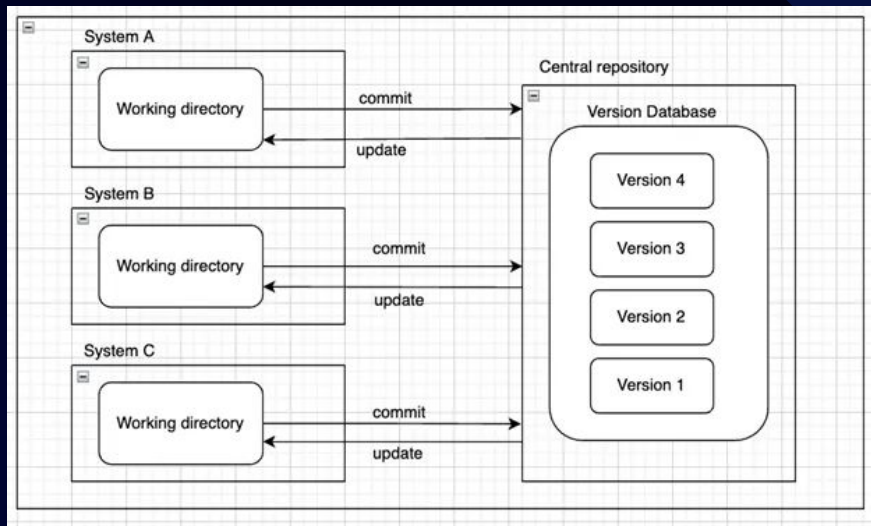
Types of Version Control Systems

- Version Control Systems help manage changes in code
- **There are two main types:**
 - **Centralized VCS**
 - **Distributed VCS**



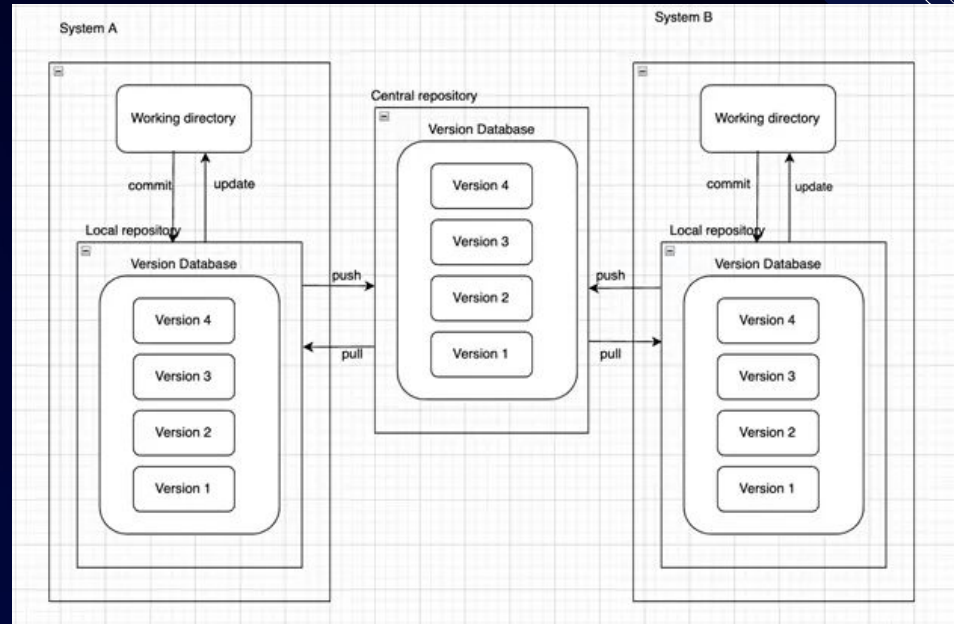
Centralized VCS

- One central server stores all project files
- Developers connect to this server to work
- Example workflow:
Checkout → Make changes → Commit to server
- Depends heavily on the central server



Distributed VCS

- Each user has a complete copy of the repository
- Work can be done offline
- Changes are shared between repositories
- More flexible and faster



CVCS vs DVCS

CVCS



Single point of failure

If the central server fails, all collaboration stops.



Requires internet connection

Users must be online to commit or access history.



Slower operations

Operations can be slower due to reliance on the central server.

DVCS



No single point of failure

If one repository is lost, it can be restored from others.



Supports offline work

Developers can commit and access history without an internet connection.



Faster operations

Operations are generally faster as they are local.

Pop Quiz

Q. What is a major drawback of **Centralized VCS**?

A

No collaboration

B

Single point of failure

Pop Quiz

Q. What is a major drawback of **Centralized VCS**?

A

No collaboration

B

Single point of failure



Introduction to Git

Overview of Git

- **History of Git:** Git was created by Linus Torvalds in 2005 to manage the Linux kernel development.
- **Comparison with Other VCS:** Git is a distributed VCS, unlike centralized systems like SVN. It provides better performance, flexibility, and collaboration.



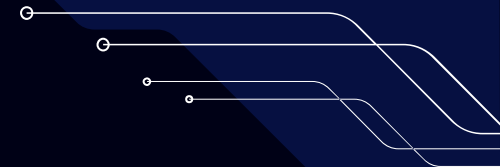
Unique Features and Benefits of Git

- **Distributed Version Control:** Git's distributed architecture allows every user to have a complete copy of the repository, enhancing collaboration and resilience, and enabling work locally without needing constant internet access.
- **Branching:** Git enables easy creation and management of branches, allowing multiple features and bug fixes to be developed concurrently.
- **Merging:** Powerful merging capabilities facilitate the integration of changes from different branches.



Setting Up Git On Windows

- Installation on Windows:
Download the installer from Git's official website and follow the setup instructions.
[LINK](#)
- When you've successfully started the installer, you should see the Git Setup wizard screen. Follow the Next and Finish prompts to complete the installation. The default options are pretty sensible for most users.



Setting Up Git On Mac

- Download the latest Git for Mac installer.

[LINK](#)

- Follow the prompts to install Git.
- Open a terminal and verify the installation was successful by typing `git --version`:

```
$ git --version  
git version 2.9.2
```



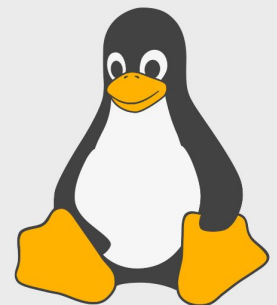
Setting Up Git On Linux

- From your shell, install Git using apt-get:

```
$ sudo apt-get update  
$ sudo apt-get install git
```

- Follow the prompts to install Git.
- Verify the installation was successful by typing `git --version`:

```
$ git --version  
git version 2.9.2
```



Linux™

Pop Quiz

Q. Who created Git?

A

Elon Musk

B

Linus Torvalds

Pop Quiz

Q. Who created Git?

A

Elon Musk

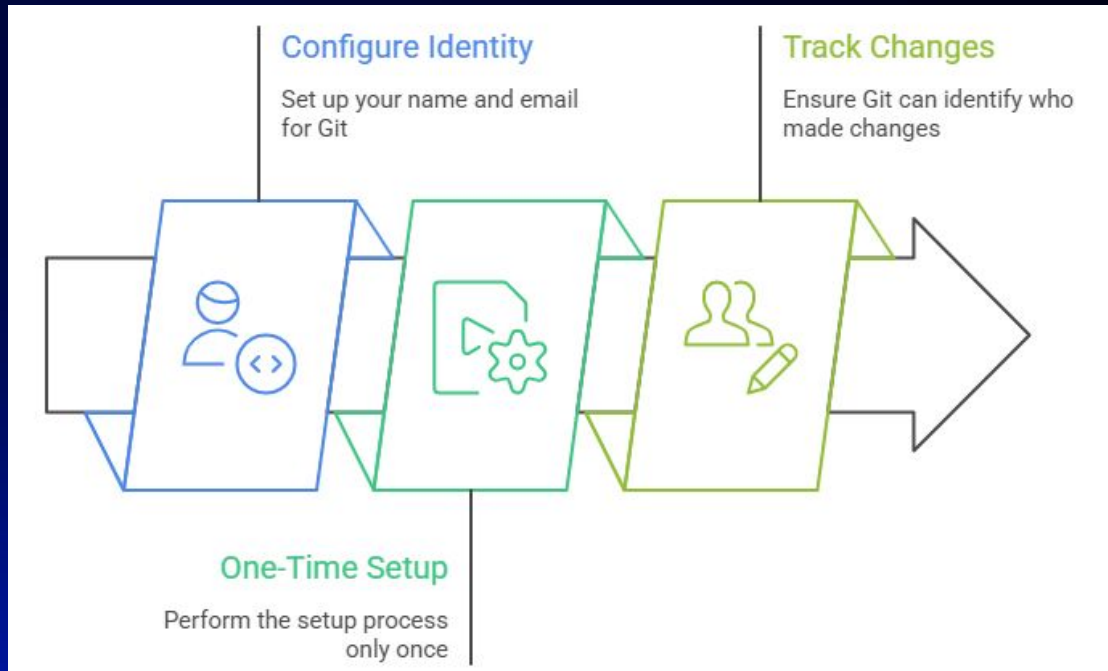
B

Linus Torvalds



Git Setup (One-Time Configuration)

Git Setup (One-Time Configuration)



Configuring Git with Username and Email

- Open a Command Prompt (or Git Bash if during installation you elected not to use Git from the Windows Command Prompt).
- Global Configuration:
Set the global username and email for commits using the commands:

```
git config --global user.name "Your Name"
```

and

```
git config --global user.email "youremail@example.com".
```
- Verification:
Verify the configuration with `git config --list` to ensure the information is correct.



Why Configuration is Important



The top corners of the slide feature decorative white circuit-like lines on a dark blue background. These lines start from the left and right edges, curve upwards, and then branch out horizontally, ending in small circles. The lines are more densely packed on the left side and more spread out on the right side.

Hands-On | Git Setup (One-Time Configuration)

Pop Quiz

Q. What does the --global flag mean?

A

**Applies to all repositories on
your system**

B

Applies only to one file

Pop Quiz

Q. What does the --global flag mean?

A

**Applies to all repositories on
your system**

B

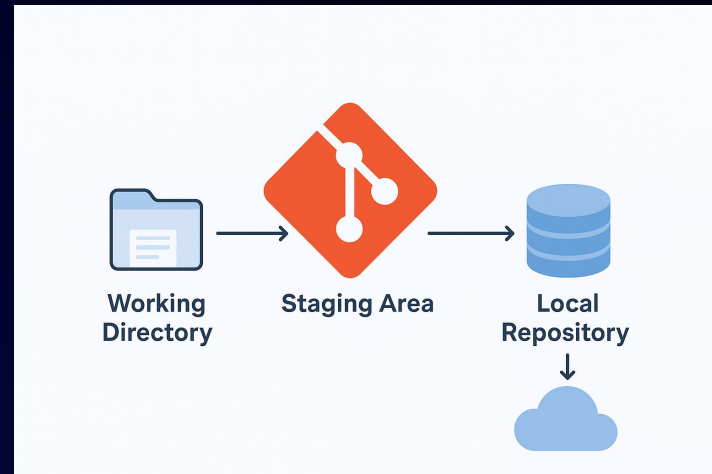
Applies only to one file



Git Architecture (Simple Mental Model)

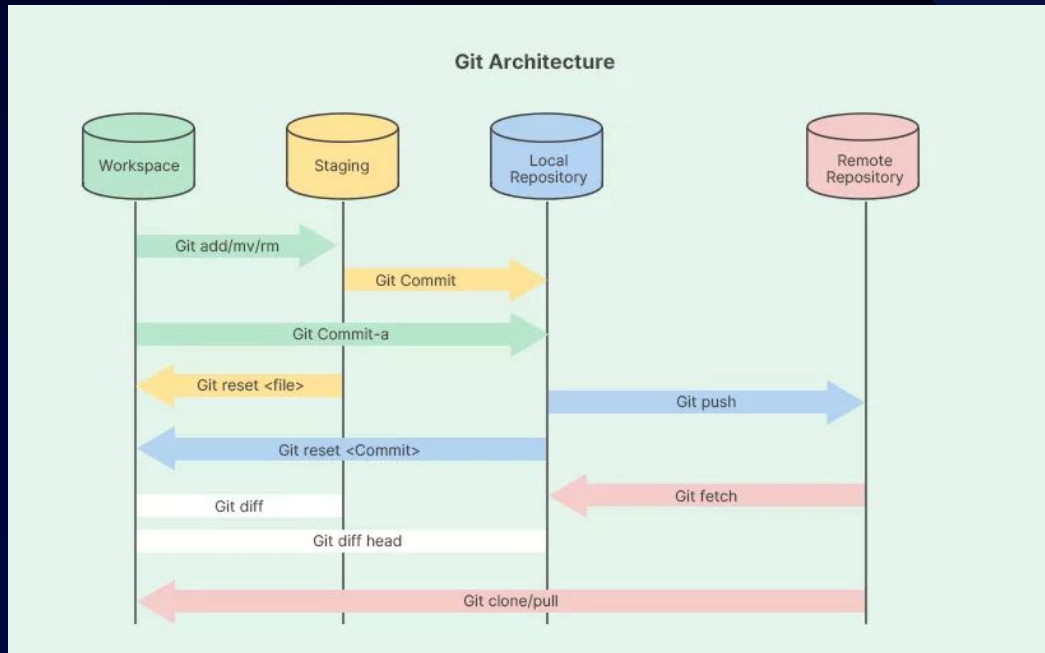
Git Architecture (Simple Mental Model)

- Git works in three main areas
- Helps you manage changes step by step



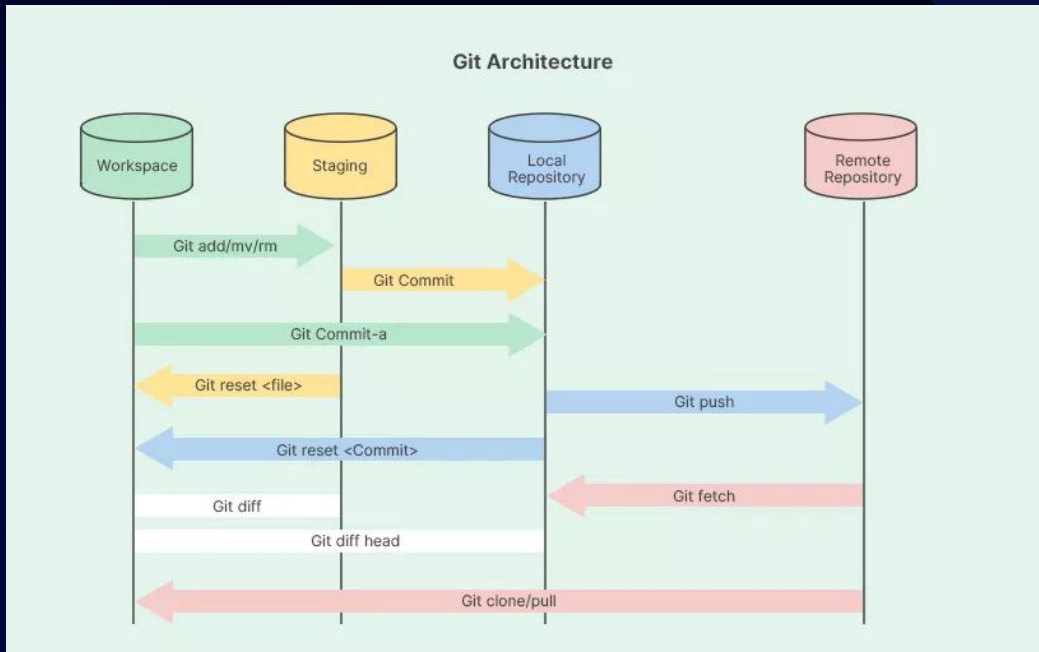
Working Directory

- This is where your files live
- You create, edit, and delete files here
- Changes are not tracked yet by Git



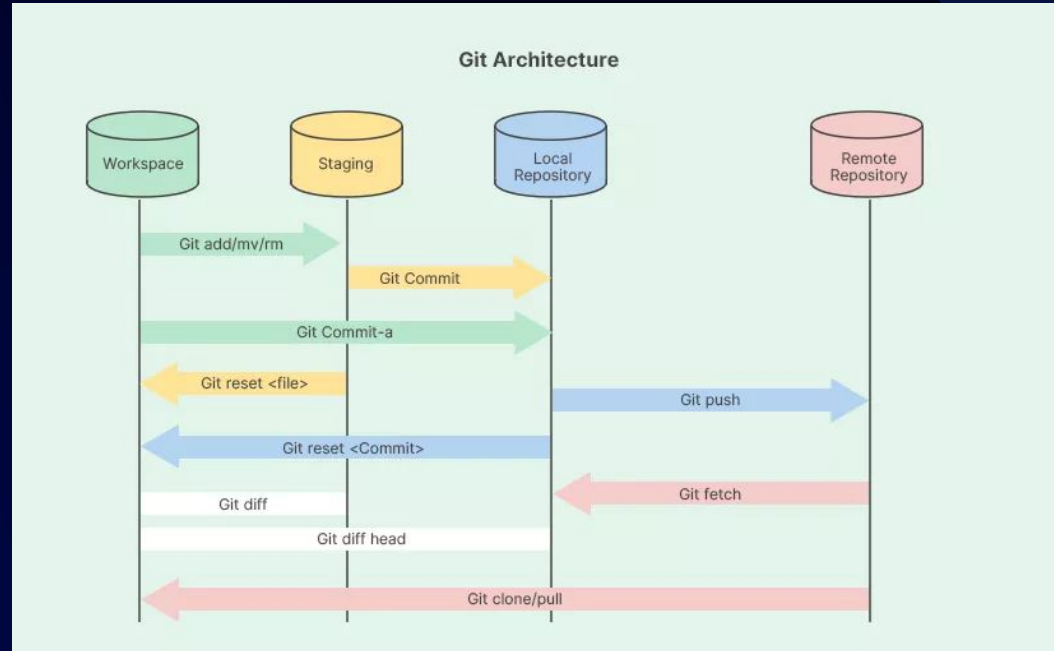
Staging Area

- Also called the “index”
- You add files here before committing
- Acts like a preparation zone
- Only staged changes will be saved



Repository

- Stores all committed changes
- Keeps the complete history of your project
- Each commit is a saved version



Pop Quiz

Q. What is the purpose of the Staging Area?

A

To store final history

B

**To prepare changes before
committing**

Pop Quiz

Q. What is the purpose of the Staging Area?

A

To store final history

B

**To prepare changes before
committing**



Initialize and Track a Project (Hands-On)

Initialize a New Git Repository

- Open your terminal or command prompt.
- Navigate to your project directory using the `cd` command.

For example:



```
cd path/to/your/project
```

- Once you're inside the project directory, run the following command to initialize a new Git repository:



```
git init
```


Initialize a New Git Repository

```
git init
```

- This command creates a hidden `.git` directory in your project folder, which contains all the necessary Git configuration.



Initialize a New Git Repository

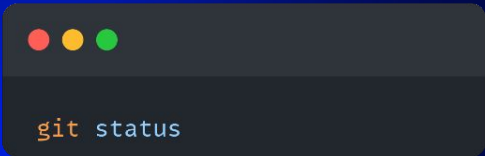
Create Sample Files

Let's create a few sample files. You can use any text editor or IDE to do this. Here's an example:

- Create an index.html file for your homepage.
- Add a style.css file for styling.
- Optionally, include an images folder with some placeholder images.
- Feel free to add more files based on your portfolio's structure.

Check Repository Status:

After creating or placing your sample files, run the following command to check the status of your repository

A dark-themed terminal window with three colored window control buttons (red, yellow, green) in the top-left corner. The text 'git status' is displayed in a light-colored monospace font.

```
git status
```

Initialize a New Git Repository

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.22631.3880]
(c) Microsoft Corporation. All rights reserved.

C:\Users\yash9\Desktop\New folder\Portfolio>git init
Initialized empty Git repository in C:/Users/yash9/Desktop/New folder/Portfolio/.git/

C:\Users\yash9\Desktop\New folder\Portfolio>git status
On branch master

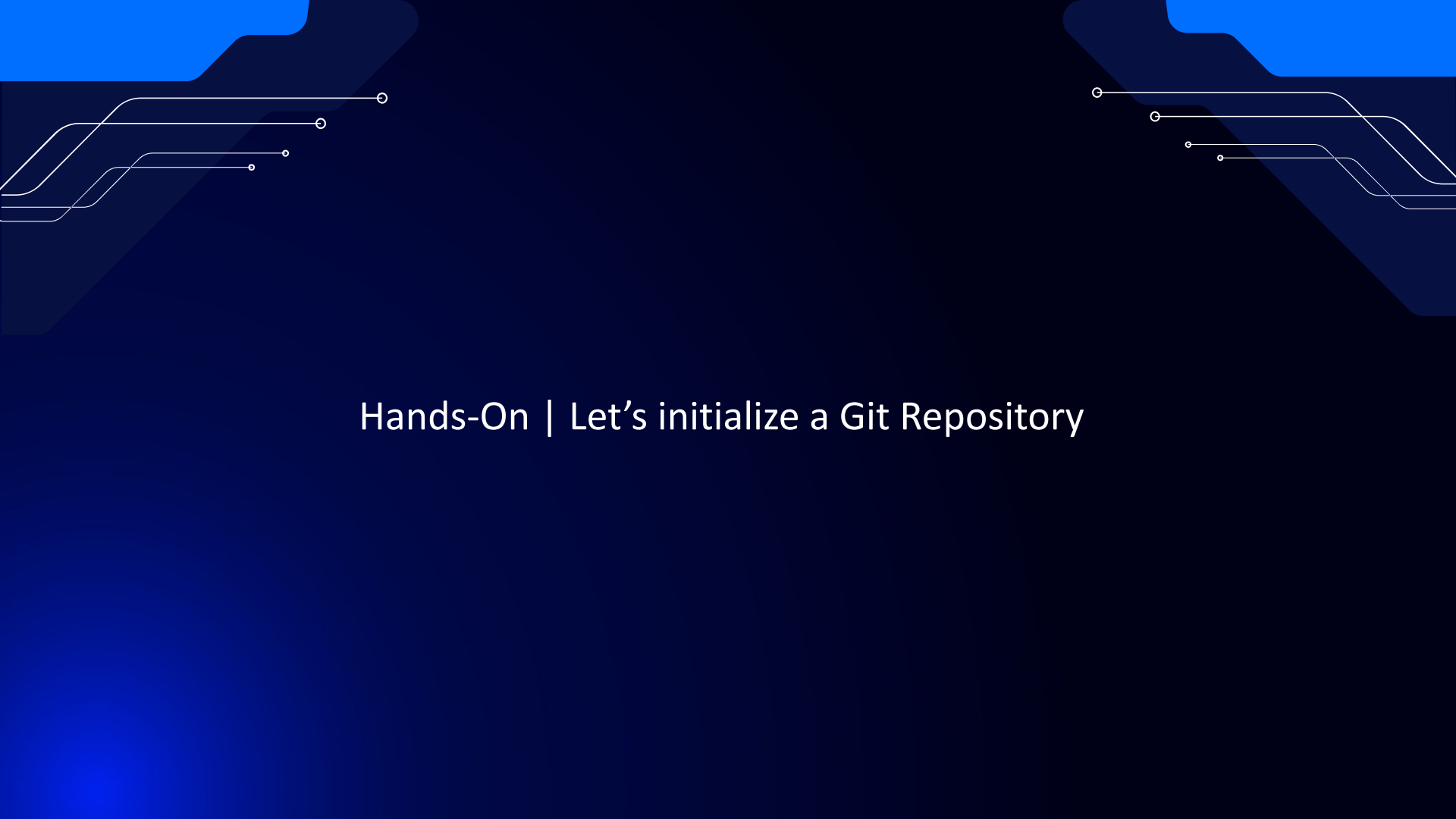
No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html
        style.css

nothing added to commit but untracked files present (use "git add" to track)
C:\Users\yash9\Desktop\New folder\Portfolio>
```

Git will show you information about untracked files, modified files, and the current branch.

The files in the red colour are untracked files

The background is a dark blue gradient. In the top left and top right corners, there are decorative elements consisting of white lines and small circles. These lines are of varying lengths and are connected to small white circles, creating a network-like or circuit-like pattern. The lines are white and stand out against the dark blue background.

Hands-On | Let's initialize a Git Repository

Pop Quiz

Q. What command initializes a new Git repository in a project directory

A

git init

B

git create

Pop Quiz

Q. What command initializes a new Git repository in a project directory

A

git init

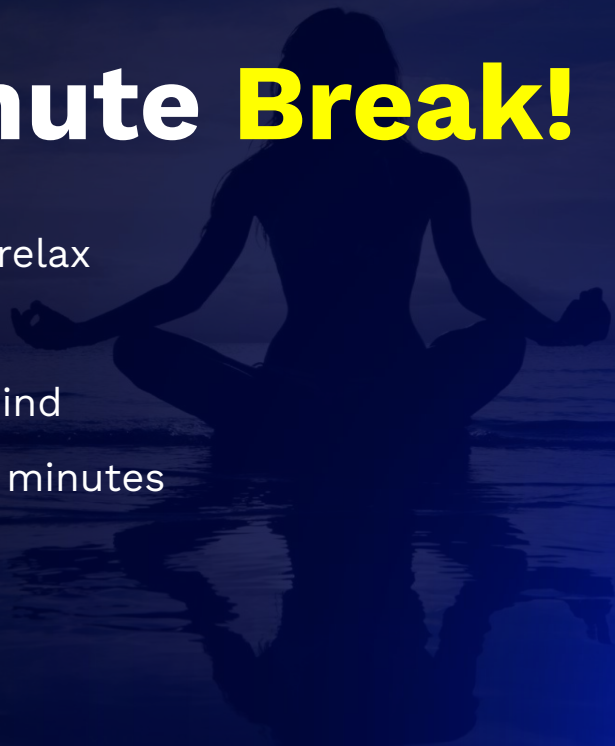
B

git create



Take A 5-Minute **Break!**

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes

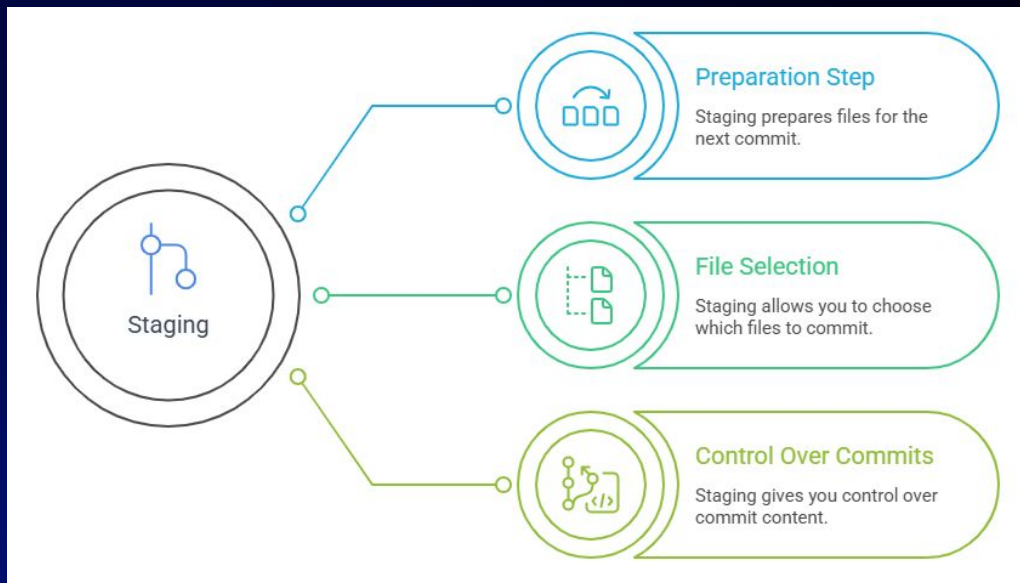




Add and Commit Changes (Core Workflow)

What is Staging?

- Staging = selecting files to include in the next commit
- Acts like a preparation step
- You control what goes into a commit



Adding Files to the Staging Area

- After creating or placing your sample files in the project directory, you'll want to add them to the staging area. The staging area is where you prepare files for committing.
- To add specific files, use the following command (replace <filename> with the actual filename):
`git add <filename>`
- To add all files (including any new files), use:
`git add .`
- The git add command stages changes for the next commit.

Writing Good Commit Messages



Committing Changes:

- Once your files are in the staging area, it's time to commit them. A commit represents a snapshot of your project at a specific point in time.
- Use the following command to commit your changes along with a meaningful message:
`git commit -m "Initial commit"`
- Replace "Initial commit" with a description that reflects what you've done in this commit. For example, if you've added your homepage and styling, you could use "Added initial HTML and CSS files" as your commit message.

```
C:\Users\yash9\Desktop\New folder\Portfolio>git add index.html
```

```
C:\Users\yash9\Desktop\New folder\Portfolio>git commit -m "Added index.html file"
[master (root-commit) 4353896] Added index.html file
1 file changed, 12 insertions(+)
create mode 100644 index.html
```



Hands-On | Add and Commit Changes (Core Workflow)

Pop Quiz

Q. What is the role of the staging area?

A

Stores final history

B

Prepares files before committing

Pop Quiz

Q. What is the role of the staging area?

A

Stores final history

B

Prepares files before committing



View History

Viewing Commit History:

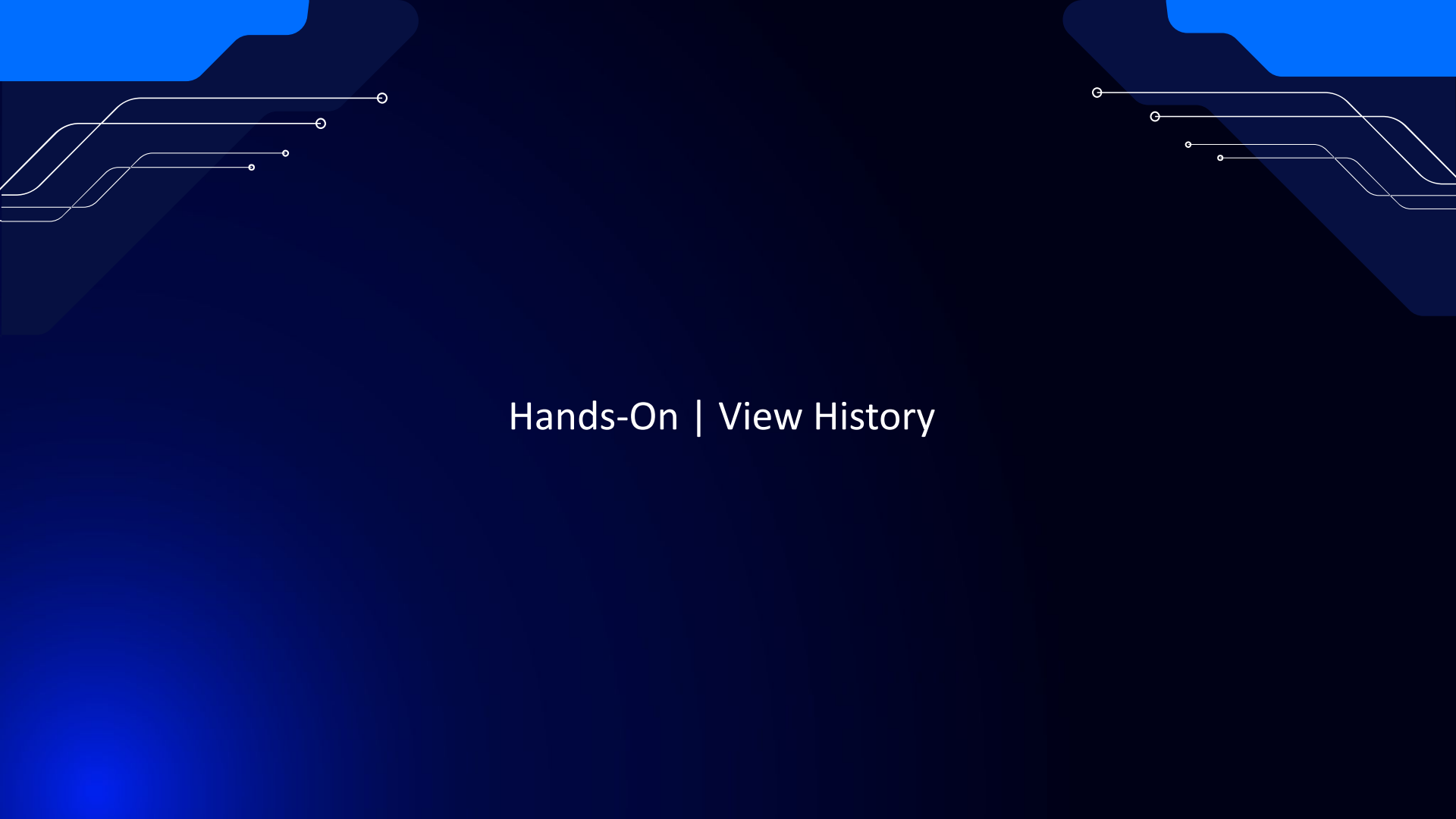
- To verify that your commit was successful, run:

```
git log
```

```
C:\Users\yash9\Desktop\New folder\Portfolio>git log
commit 435389691b769271106a3d59f9c8eb21e25200e7 (HEAD -> master)
Author: yashsaini7060 <yk18052000@gmail.com>
Date: Tue Jul 23 17:39:41 2024 +0530

    Added index.html file

C:\Users\yash9\Desktop\New folder\Portfolio>|
```

The background is a dark blue gradient. In the top-left and top-right corners, there are decorative elements consisting of white lines and small circles. These lines are of varying lengths and are arranged in a way that suggests a network or a series of connections. The circles are small and white, acting as nodes or endpoints for the lines.

Hands-On | View History

Pop Quiz

Q. What does git log display?

A

Commit history with details

B

Current file changes

Pop Quiz

Q. What does git log display?

A

Commit history with details

B

Current file changes



gitignore (Important Practice)

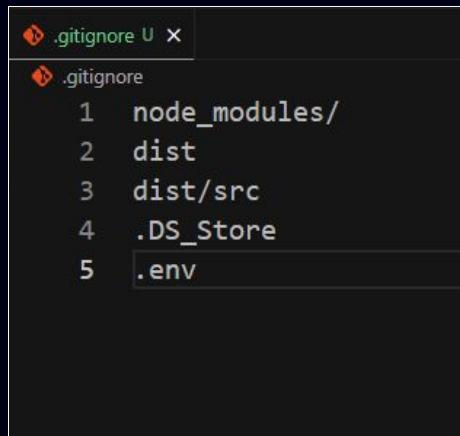
Introduction to .gitignore Files

- The .gitignore file is like a project bouncer—it decides who gets into your Git party and who waits outside.
- Its mission: Keep the noise out! Noise includes files you don't want cluttering up your version control history.

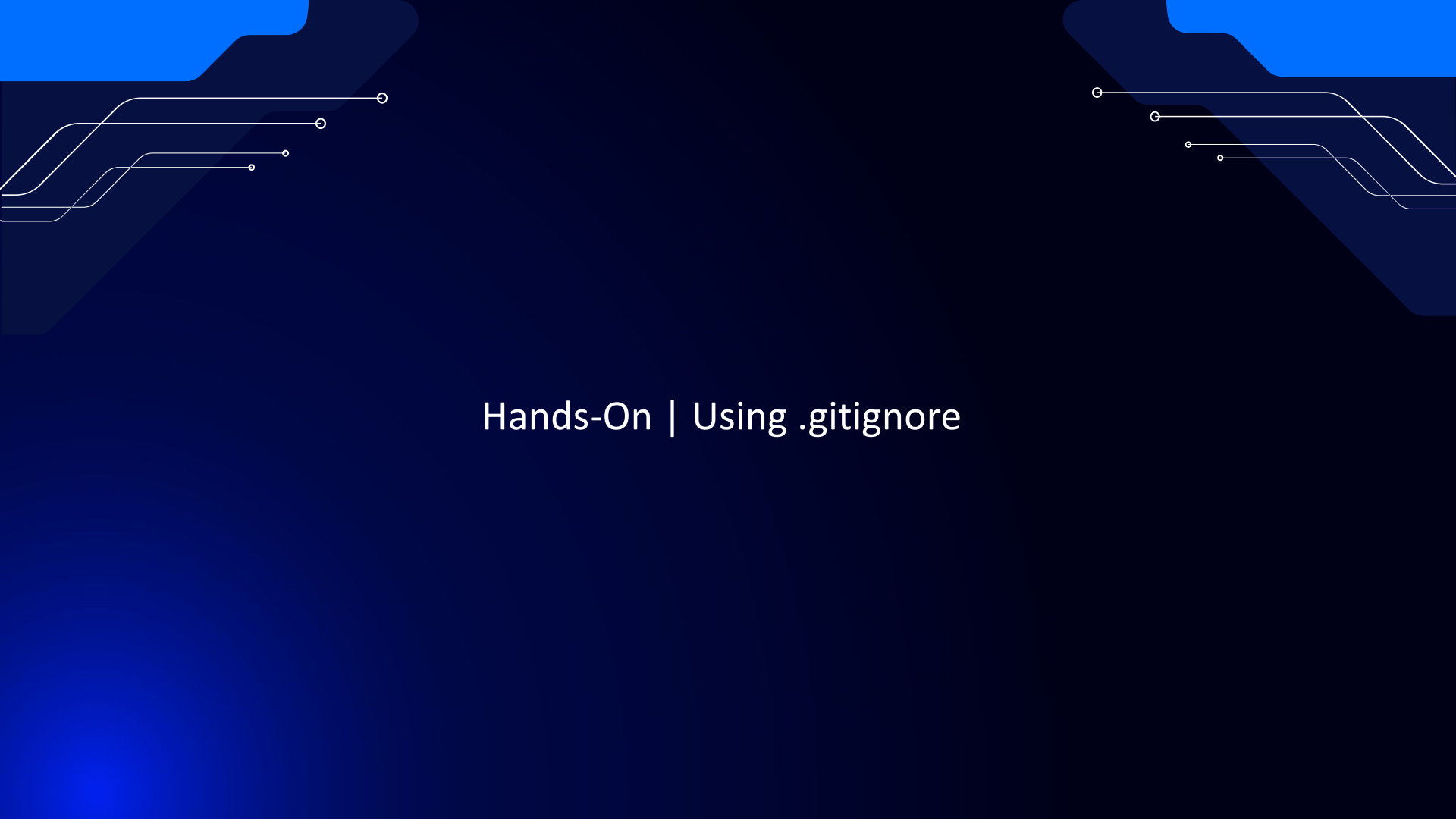


Creating a .gitignore File

- Open your trusty text editor (Visual Studio Code, Sublime Text, or even Notepad) or fire up the terminal.
- Navigate to your project's root directory (where your main project files reside).
- Craft Your `.gitignore`:
 - Create a new file named `.gitignore`.
 - Add patterns for files or directories you want Git to ignore.
 - Each pattern goes on a separate line.
 - Examples:
 - `*.pyc`: Ignore compiled Python files.
 - `.DS_Store`: Exclude macOS-specific metadata files.
 - `node_modules/`: Skip the entire Node.js dependencies folder.

A screenshot of a code editor window showing a file named `.gitignore`. The file contains five lines of text, each on a new line: `node_modules/`, `dist`, `dist/src`, `.DS_Store`, and `.env`. The lines are numbered 1 through 5 on the left side of the editor. The editor has a dark theme and a tab at the top labeled `.gitignore U x`.

```
.gitignore
1 node_modules/
2 dist
3 dist/src
4 .DS_Store
5 .env
```

The background is a dark blue gradient. In the top-left and top-right corners, there are decorative elements consisting of white lines and small circles. These lines are of varying lengths and are connected to small white circles, creating a circuit-like or network-like pattern. The lines are white and stand out against the dark blue background.

Hands-On | Using .gitignore

Pop Quiz

Q. What happens if a file is listed in .gitignore?

A

It will be committed automatically

B

It will not be tracked by Git

Pop Quiz

Q. What happens if a file is listed in .gitignore?

A

It will be committed automatically

B

It will not be tracked by Git

Summary

In this lecture, we have covered:

- Understood the importance of version control and its role in tracking changes
- Learned types of version control: centralized and distributed
- Got introduced to Git as a distributed VCS
- Set up Git with basic configuration
- Explored Git architecture (working directory, staging, repository)
- Initialized and started tracking a project
- Learned to add and commit changes
- Viewed commit history
- Used .gitignore to exclude unnecessary files

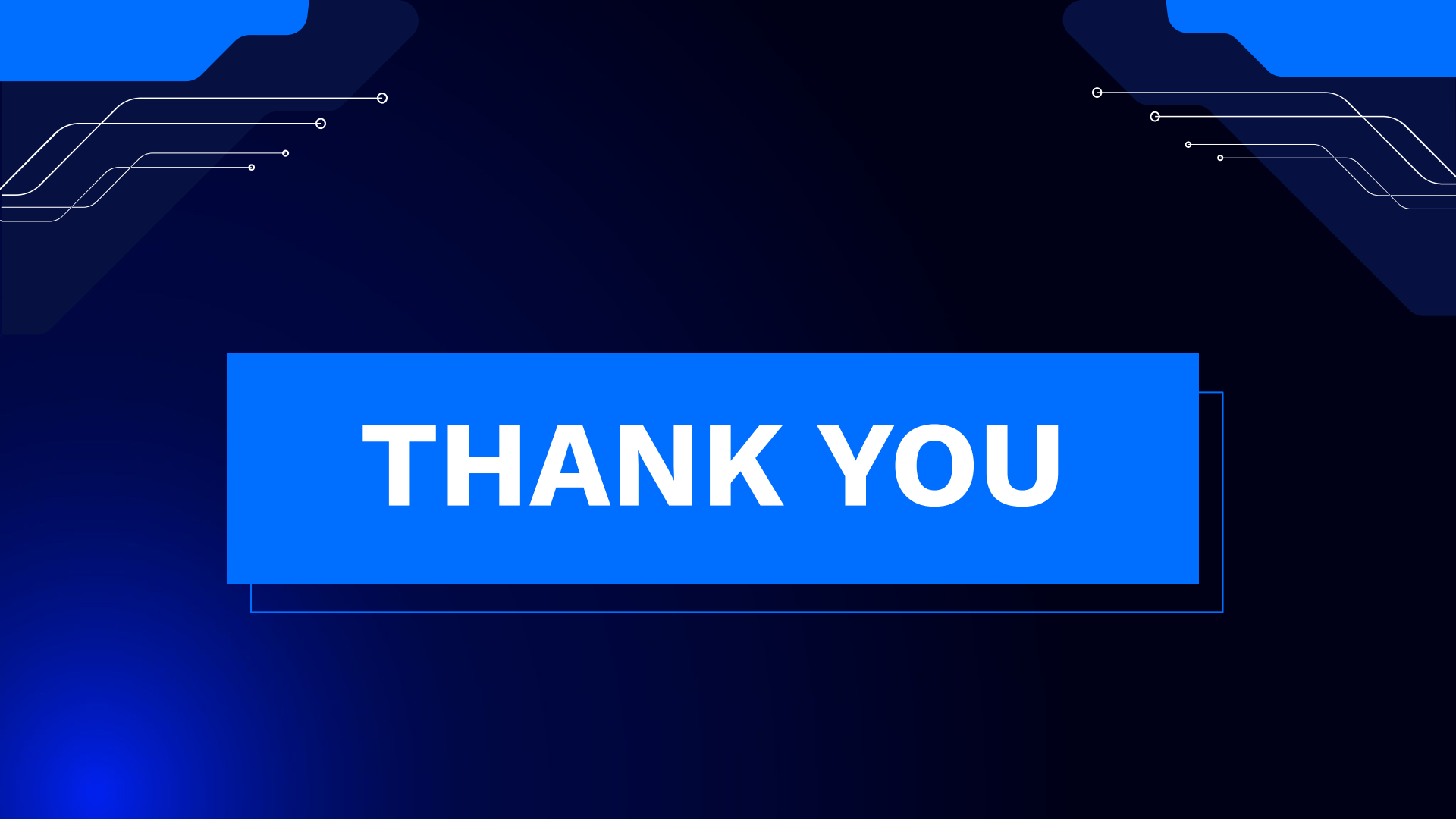




DOUBT RESOLUTION

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.



THANK YOU